

Master's end-of-studies internship report

Master 2 — Data Science and Engineering

Decouple to harden: migrating and industrialising data pipelines at Qonto

Student Ayoub ELHARRAN

Company Qonto (Olinda SAS)

Period April – October 2026

Company supervisor Michael Ourch
Senior Data Engineer, Data Platform
michael.ourch@qonto.com

University tutor Maxime Berar

Abstract

This end-of-studies internship took place within Qonto's *Data Platform* team, a European neobank and payment service provider (PSP). It focused on hardening and industrialising data pipelines orchestrated by Apache Airflow. Three projects are presented. The first is the migration of Notion data ingestion to the *dlt* (data load tool) framework with a direct Snowflake destination: it was triggered by a *silent data-truncation* incident caused by an API change, and its core challenge was achieving *byte-for-byte* parity with the legacy system so that the production cutover would be downtime-free and transparent to the hundreds of downstream dbt models. The second project replaced personal-access-token authentication (14 manually rotated PATs relying on a brittle pooling mechanism) with a JWT-based *Connected App* (a single secret), while moving the Tableau pipelines out of Airflow's shared Docker base image. The third, being finalised, implements the *CESOP Netherlands* regulatory transmission chain to the government Digipoort gateway over HTTPS/mTLS with a PKIoverheid certificate, as two DAGs (a quarterly submitter and an asynchronous receipt poller). The common thread of the first two projects is *decoupling* a shared infrastructure dependency, identified as the true source of fragility, before fixing the business problem itself. Results include eliminating silent truncation by construction, removing all manual token rotation, and substantially reducing legacy code and the number of Airflow tasks.

Keywords: data engineering, Apache Airflow, dlt, Snowflake, dbt, EL ingestion, orchestration, JWT authentication, mTLS/PKI, regulatory compliance (CESOP), zero-downtime migration.

Résumé

Ce stage de fin d'études s'est déroulé au sein de l'équipe *Data Platform* de Qonto, néobanque européenne et prestataire de services de paiement (PSP). Il portait sur la fiabilisation et l'industrialisation de pipelines de données orchestrés par Apache Airflow. Trois projets y sont présentés. Le premier est la migration de l'ingestion des données Notion vers le framework *dlt* (data load tool) avec écriture directe dans Snowflake : le déclencheur était un incident de *troncature silencieuse* causé par un changement d'API, et la difficulté centrale a été d'obtenir une parité *octet-à-octet* avec l'ancien système pour réaliser une bascule de production sans interruption ni régression sur les modèles dbt en aval. Le deuxième projet a remplacé l'authentification par jetons personnels (14 PAT à rotation manuelle, avec un mécanisme de pool fragile) par une *Connected App* en JWT (un unique secret), tout en migrant les pipelines Tableau hors de l'image Docker partagée d'Airflow. Le troisième, en cours de finalisation, met en place la chaîne de transmission réglementaire *CESOP Pays-Bas* vers la passerelle gouvernementale Digipoort en HTTPS/mTLS avec certificat PKIoverheid, sous la forme de deux DAGs. Le fil conducteur des deux premiers projets est le *découplage* d'une dépendance d'infrastructure partagée, identifiée

comme la véritable cause de fragilité, avant la correction du problème métier.

Mots-clés : ingénierie des données, Apache Airflow, dlt, Snowflake, dbt, ingestion EL, orchestration, authentification JWT, mTLS/PKI, conformité réglementaire (CESOP), migration sans interruption.

Acknowledgements

I would like to thank the entire *Data Platform* team at Qonto for their welcome and the trust they placed in me throughout this internship.

My thanks go first to my company supervisor, Michael Ourch, for the mentoring, the quality of the code reviews, and the latitude I was given to take ownership of high-impact, production-facing topics.

I also thank the team's engineers for their availability, their demanding reviews, and the technical discussions that shaped each of the projects presented in this report, as well as the *Regulatory* and *SRE* teams with whom I collaborated on the CESOP project.

Finally, I express my gratitude to Maxime Berar, my university tutor, and to the teaching team of the Université de Rouen Normandie — UFR of Sciences and Technology, for their guidance and advice during this final year of the master's programme.

Contents

Abstract / Résumé	i
Acknowledgements	iii
Glossary & acronyms	vi
Introduction	1
1 Internship context	3
1.1 The company: Qonto	3
1.2 The Data Platform team and the working method: the Qonto Way	3
1.3 The technical ecosystem	5
1.3.1 Airflow and config-driven DAG generation	6
1.3.2 The <i>Job Orchestrator</i> : moving a job out of the shared image	6
1.3.3 Data contracts: Qontract	7
1.4 My position and assignments	8
2 State of the art and problem statement	9
2.1 Integration paradigms: from ETL to EL	9
2.2 Ingestion frameworks: from the in-house operator to the declarative framework	9
2.3 Orchestration and infrastructure coupling	10
2.4 Machine-to-machine authentication	10
2.5 Warehouse loading: staging, COPY and atomic swap	11
2.6 Regulatory reporting and data transmission	11
2.7 Problem statement and positioning	11
3 Migrating Notion ingestion to dlt	12
3.1 The legacy	12
3.2 The trigger: a silent-truncation incident	12
3.3 Options considered and decision	13
3.4 Implementation: the dlt architecture	13
3.5 The real challenge: byte-for-byte parity with the legacy	14
3.6 Two mechanisms re-implemented around dlt	15
3.7 The zero-downtime production cutover	15
4 Migrating the Tableau DAGs and JWT authentication	17
4.1 The legacy	17
4.2 The central problem: authentication	17
4.2.1 The legacy XCom mechanism	17

4.3	Solution: dedicated pods and a JWT Connected App	18
4.4	How the JWT works	18
4.5	Difficulties encountered	19
4.6	Decisions and trade-offs	19
5	CESOP Netherlands regulatory transmission chain	21
5.1	Regulatory context	21
5.2	The Dutch specificity	21
5.3	Architecture: two DAGs and a state table	22
5.4	Implementation	22
5.5	Sticking points resolved	23
6	Assessment and skills acquired	24
6.1	A common pattern: diagnose, decouple, harden	24
6.2	Technical skills	24
6.3	Methodological skills	25
6.4	Transferable skills	25
6.5	Value for the company	25
	Conclusion and perspectives	26
	Bibliography	27
A	Detailed presentation of Qonto	28
B	Technical environment and repositories	29
C	Main contributions (Pull Requests)	30
D	Code excerpts	31

Glossary & acronyms

Airflow

Open-source workflow orchestrator. A pipeline is described as a **DAG**.

DAG

Directed Acyclic Graph: a scheduled pipeline, a set of tasks linked by dependencies.

dbt

data build tool: SQL transformation tool running downstream of the data warehouse.

dlt

data load tool: a Python **EL** ingestion library (extract + load) with automatic normalisation, staging and loading.

EL / ETL

Data-integration paradigms: *Extract–Load* (transformation deferred downstream) vs *Extract–Transform–Load*.

Snowflake

Cloud data warehouse, the target of every ingestion.

S3

Amazon Web Services object storage, used as a *staging* area.

Data contract (Qontract)

A YAML file declaring the fields, types and key of an ingested table.

Job Orchestrator

Qonto-internal pattern: a job is declared in a `schedule.yaml` and its Airflow DAG is *auto-generated*, in a repository and Docker image dedicated to that job.

PAT

Personal Access Token: a personal token with limited lifetime and manual rotation.

JWT

JSON Web Token: a signed token, used here via a Tableau *Connected App* for authentication.

Connected App

A trusted application declared on the Tableau side, enabling signed-JWT authentication instead of PATs.

mTLS

mutual TLS: authentication in which both client and server present a certificate.

PKIoverheid

The Dutch government's public-key infrastructure; issues trusted X.509 certificates.

Digipoort

The Dutch government's file-exchange gateway (operated by Logius).

FTPS

FTP over TLS (RFC 4217) — not to be confused with SFTP (FTP over SSH).

CESOP

Central Electronic System of Payment information: the EU system for collecting cross-border payment data (VAT anti-fraud).

PSP

Payment Service Provider (Qonto's status).

PR

Pull Request: a proposed code change submitted for review.

VARIANT

Snowflake's semi-structured type (stores native JSON / arrays / objects).

Introduction

Data sits at the heart of how a financial company operates: it feeds steering, fraud detection, customer relationship management and, increasingly, regulatory obligations. At Qonto, a European neobank and payment service provider, hundreds of data flows converge every night into a Snowflake warehouse, where they are transformed and then exposed to business teams. These flows are orchestrated by Apache Airflow. Their reliability directly conditions the quality of downstream decisions: a wrong figure, an incomplete table or an unrefreshed dashboard all have concrete consequences.

My end-of-studies internship took place within the *Data Platform* team, whose mission is precisely to build and maintain this ingestion, transformation and delivery infrastructure. The initial topic was about hardening existing pipelines that were quickly crystallised around a recurring observation: several jobs historically written *inside* the monolithic qonto-airflow repository and its shared Docker image suffered from the same structural weakness. That image, common to every pipeline, made any library upgrade risky (it could affect all flows) and concealed silent failures. The problem statement of this internship can therefore be phrased as follows:

How can we harden and industrialise business-critical data pipelines, without any service interruption for their consumers, when the true source of fragility is not the business logic but a shared infrastructure dependency?

This report presents three projects carried out in response to that question. The first two share a common pattern: diagnosing that the real constraint is the coupling to the Airflow image, *decoupling* the job into a dedicated repository and image, then taking the opportunity to remove a brittle legacy mechanism. The first project migrates Notion ingestion to the *dlt* framework in response to a silent-truncation incident; the second replaces hard-to-maintain personal-token authentication with a JWT-based *Connected App* for the Tableau pipelines. The third project, of a different nature, consists in building a brand-new regulatory transmission chain (the CESOP filing for the Netherlands) to a government gateway that imposes a specific exchange protocol and authentication scheme.

The report is organised as follows. **Chapter 1** introduces the company, the team and the technical ecosystem in which the projects take place, together with the working method. **Chapter 2** reviews the state of the art of the relevant ingestion, orchestration and authentication approaches, and formalises the common problem. **Chapters 3, 4 and 5** detail the three projects, each following the same outline: the legacy, the problem, the options considered and the decision, the implementation, the difficulties encountered and the evaluation of results. **Chapter 6** draws up an assessment of the skills acquired. The conclusion revisits the main lessons and outlines perspectives.

Throughout the report, particular care is taken to distinguish teamwork from my *personal*

contribution, as recommended by the assessment guidelines, and to *measure*, wherever possible, the effects of the changes introduced.

Internship context

This chapter introduces the host company, the team I worked in, the technical ecosystem underpinning the projects, and the working method. It provides the background needed to understand the following chapters; a more detailed company presentation is given in Appendix A.

1.1 The company: Qonto

Qonto is a European neobank for freelancers, very small businesses and SMEs. Founded in 2017 by Alexandre Prot and Steve Anavi, it is operated by the company Olinda SAS and offers a business account together with financial-management services: payment cards, transfers, invoicing, pre-accounting and steering tools [13, 12]. At the time of the internship, Qonto reports on the order of 600,000 customers and operates in eight European countries: France, Germany, Italy, Spain, Austria, Belgium, the Netherlands and Portugal, following an expansion into four new markets in 2024 [12].

On the regulatory side, Qonto operates as a *payment institution* (Payment Service Provider, PSP). This status is not a mere legal detail: it entails regulatory-reporting obligations, including the European CESOP filing addressed in Chapter 5 and demands a high level of rigour on data reliability and traceability. The opening of a Dutch IBAN in late 2025 is precisely what made Qonto liable for the CESOP filing in the Netherlands.

1.2 The Data Platform team and the working method: the Qonto Way

I completed my internship within the *Data Platform* team, part of the Data department. Its mission is to provide business and other technical teams with a reliable platform for ingesting, transforming and delivering data. It owns the “plumbing”: moving data from dozens of sources into the warehouse, guaranteeing its freshness and accuracy, and exposing the results.

Each initiative is run as a Notion *project card* following the *Qonto Way*, a structured lifecycle in three phases (Figure 1.1). **Value Analysis** (VA) frames the problem: an *Intro & Context*, the *Definition of Done*, and the *Goals*. **Value Engineering** (VE) designs the solution: a *Visual Spec*, the *Hard Points* (the open questions or blockers to resolve before building), the *Solutions*, and a *Dive-in* holding the deep implementation notes. **Build** then executes the work as a task kanban (*Not started* / *In progress* / *Done*). I followed this method for all three projects; the card in Figure 1.1 is the real one for the Tableau migration (Chapter 4).

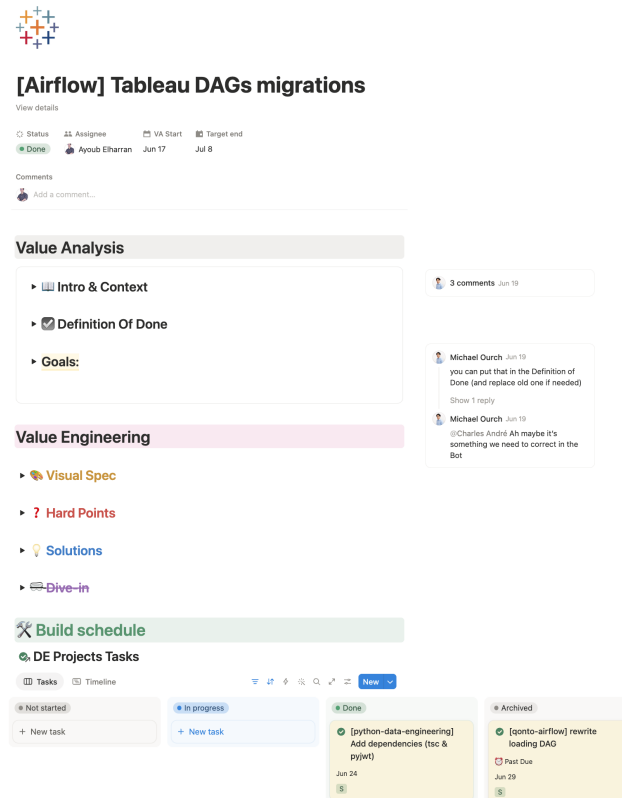


Figure 1.1: The Notion project card that structures the work following the “the Qonto Way”.

At team level, a *Delivery Board* tracks every project across the same lifecycle stages, *Value Analysis* → *Value Engineering* → *Build* → *Value Validation*, each with a progress percentage, its blockers and a target date, and reviewed in a daily stand-up (Figure 1.2). This gave a shared, at-a-glance picture of where each initiative stood. In practice the method strongly structured my work: it forced me to pin down the *Definition of Done* before coding, to surface the *Hard Points* early, and to make trade-offs explicit in the *Solutions* and *Dive-in* sections, a discipline reflected throughout this report.

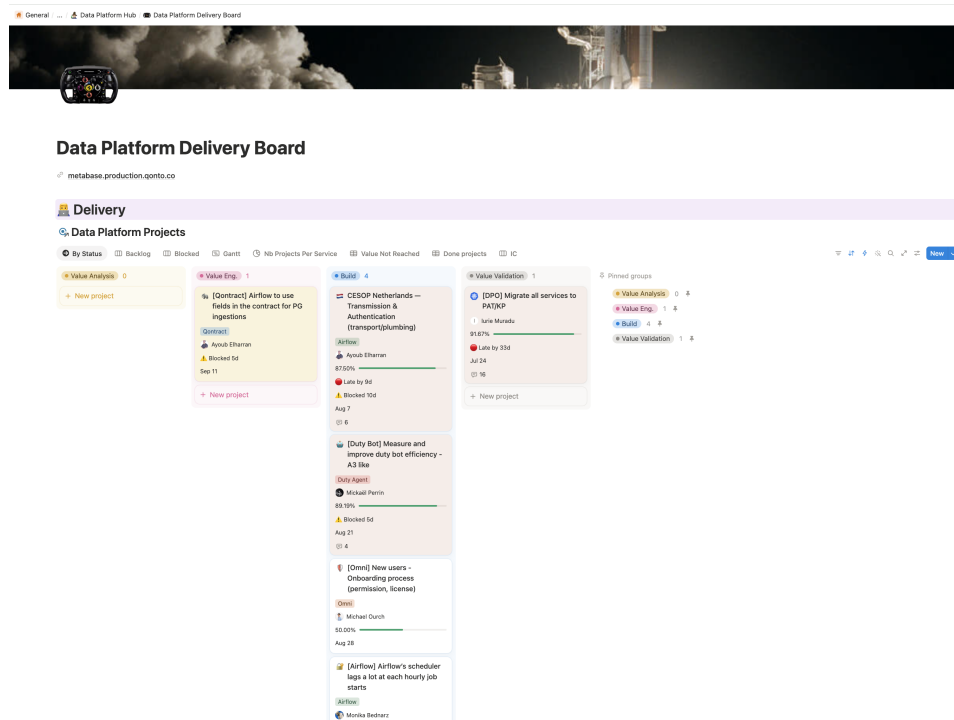


Figure 1.2: The Data Platform Delivery Board during the internship: ongoing projects arranged by lifecycle stage (Value Analysis → Value Engineering → Build → Value Validation), each with a progress percentage, its blockers and a target date. The CESOP project (Build column) is assigned to the author.

On the engineering side, work goes through *Pull Requests* (PRs) systematically reviewed by one or more engineers, continuous integration (tests, linting, schema validation) and a deployment across successive environments (*staging* then production). This review discipline proved decisive, in particular for the zero-downtime production cutovers described later.

1.3 The technical ecosystem

The ecosystem revolves around a few building blocks (Figure 1.3):

- **Apache Airflow** is the central orchestrator. Its code lives in a monolithic repository, `qonto-airflow`, whose distinctive feature is a *declarative* generation model: rather than hand-writing each DAG, one drops a configuration file and the corresponding DAG is produced automatically. This repository relies on a **shared Docker base image** used by all pipelines; a central point for what follows.
- **Snowflake** is the cloud data warehouse, the final target of every ingestion. Raw data lands in a RAW database, organised by schema (one per source).
- **Amazon S3** serves as a *staging* area: extractions are written there (CSV, JSONL, Parquet) before being loaded into Snowflake with a `COPY INTO` command.
- **DBT** (*data build tool*) handles downstream transformation: from the RAW tables, SQL models build the business tables consumed by analysts. A single DAG, `daily_transform_dbt`, materialises all these models with their dependencies.
- **Data contracts** (*Qcontract*) describe, as YAML files, the expected schema of each ingested table: fields, types, primary key. They are the source of truth for typing.

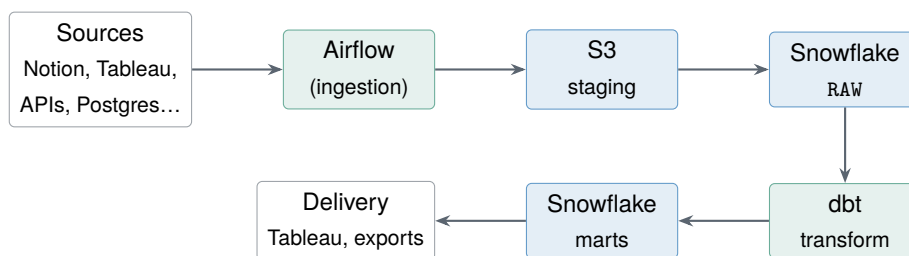


Figure 1.3: Simplified daily ETL chain of the Data Platform.

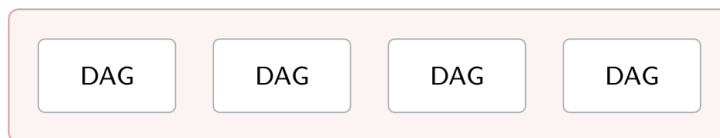
1.3.1 Airflow and config-driven DAG generation

Airflow models each pipeline as a *DAG* (Directed Acyclic Graph): a set of tasks linked by dependencies and run on a schedule. The distinctive choice in `qonto-airflow` is that these DAGs are *not* hand-written but *generated from configuration*. A pipeline is described by a small configuration file placed in a conventional folder; at parse time a generator *discovers* every such file, validates it into a typed configuration object, and emits one DAG per file. Adding a pipeline therefore means *adding a configuration*, not writing orchestration code, which is what lets the platform scale to hundreds of pipelines with little boilerplate. The flip side motivates two of my projects: because every generated DAG shares a single Docker base image, any library living in that image is coupled to *all* pipelines at once, so a version bump has a *blast radius* spanning the whole platform (Figure 1.4).

1.3.2 The *Job Orchestrator*: moving a job out of the shared image

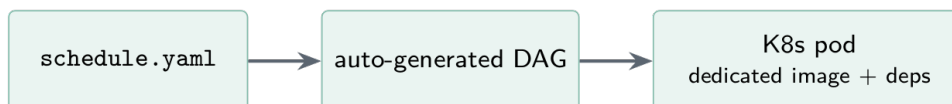
One architectural element recurs in two of the three projects: the *Job Orchestrator*. It is an internal pattern for running a job *outside* Airflow's Docker image. The principle: the job's code lives in a dedicated repository (for example `qonto-python-data-engineering` or `qonto-python-dlt`), with its own Docker image and its own dependencies; the job is simply declared in a `schedule.yaml` file, and a component (`custom_jobs_orchestrator.py`) *auto-generates* the corresponding Airflow DAG, each task running in an isolated Kubernetes *pod*. The benefit is twofold: the job's dependencies are *decoupled* from the common image, and there is no longer any DAG code to maintain in `qonto-airflow`. This mechanism is the keystone of the first two projects.

Monorepo qonto-airflow — one SHARED base image



one library bump → blast radius = every DAG at risk

Job Orchestrator — DECOUPLED



isolated dependencies · no DAG code left in the monorepo

Figure 1.4: Two execution models. In the monorepo, all DAGs share one Docker base image, so a single library bump can put every pipeline at risk. With the Job Orchestrator, a job is declared in `schedule.yaml` and its DAG is auto-generated, each task running in an isolated Kubernetes pod carrying the job's own image and dependencies.

1.3.3 Data contracts: Qontract

If config-driven generation is the *mechanism*, *Qontract* is the *configuration* that sits behind every ingestion, and the reason the platform stays consistent from source to warehouse. A Qontract is a YAML *data contract* (following the open ODCS v3.1.0 specification) that declares, for one table: its destination (Snowflake database and schema), its columns with both a *logical* and a *physical* type, its primary key, and an ingestion block telling the loader *where* to read the source and *how*. It is the single source of truth: the same file drives what is ingested, how each column is typed in Snowflake, and what the downstream dbt models and freshness tests expect. Listing 1.1 shows a condensed real contract, the Notion data_bugs board.

```

1 apiVersion: 'v3.1.0'
2 kind: 'DataContract'
3 id: 'urn:datacontract:notion:raw:notion:data_bugs'
4 version: '1.0.0'
5 status: 'active'
6 servers:
7   - { type: snowflake, database: raw, schema: notion } # destination
8 schema:
9   - name: 'data_bugs'
10     logicalType: 'object' # -> a Snowflake table
11     physicalType: 'table'
12     properties:
13       - { name: 'id', logicalType: string, physicalType: text, primaryKey: true, required: true }
14       - { name: 'Name', logicalType: string, physicalType: text }
15       - { name: 'Status', logicalType: string, physicalType: text }
16       - { name: 'created_time', logicalType: timestamp, physicalType: timestamp_ntz, required: true }
17       - { name: 'Resolved At', logicalType: date, physicalType: date }
  
```

```

18     - { name: 'archived', logicalType: boolean, physicalType: boolean, required: true }
19 customProperties:
20   - property: 'source_type'
21     value: 'notion'
22   - property: 'ingestion'
23     value:
24       page_size: 50
25       resource_id: '2e7b3f6c73bc4da19fef375e932d20e1' # which Notion data source to read

```

Listing 1.1: Condensed Qontract for the Notion data_bugs table (real contract, abridged).

Two things make the contract operational. First, the ingestion block is what the config-driven generator turns into a running job: `resource_id` identifies the exact Notion data source to query and `page_size` caps each API page. Second, the loader maps every property's declared type to a Snowflake type (Table 1.1); applying that mapping *from the contract* rather than guessing it from the data is what guarantees a stable schema at every load. Chapter 3 shows the code that performs this parsing (`build_resource_from_contract` and `yaml_to_dlt_columns`).

Contract (logical / physical)	Snowflake type	Example column
string / text	TEXT	Name, Status
timestamp / timestamp_ntz	TIMESTAMP_NTZ	created_time
date / date	DATE	Resolved At
boolean / boolean	BOOLEAN	archived
number / number	NUMBER(38,6)	—
array, object / variant	VARIANT	labels, people...

Table 1.1: How a Qontract property type maps to a Snowflake column type.

1.4 My position and assignments

Integrated into the team as a data engineer intern, I took ownership of topics end to end, from analysis to production. My three main assignments, presented in the following chapters, were: (i) migrating Notion ingestion to the dlt framework; (ii) migrating the Tableau pipelines out of the Airflow image with a move to JWT authentication; and (iii) building the CESOP regulatory transmission chain for the Netherlands.

State of the art and problem statement

The projects of this internship belong to data engineering and to software engineering applied to pipelines. This chapter situates the work with respect to the state of the art of integration paradigms, ingestion frameworks, orchestration, machine-to-machine authentication and warehouse loading. It concludes by formalising the problem common to the projects.

2.1 Integration paradigms: from ETL to EL

Data integration long followed the *ETL* paradigm (*Extract–Transform–Load*): data is extracted and transformed *before* being loaded into a target database. The advent of elastic cloud warehouses such as Snowflake or BigQuery favoured the *EL* (or *ELT*) paradigm: the raw data is *loaded first*, then transformed *inside* the warehouse using tools such as dbt [3]. Deferring transformation downstream offers several recognised advantages [1]: the raw data remains available (replayability), the transformation logic is versioned and tested like code, and extraction reduces to a faithful copy operation. Qonto’s platform follows this paradigm: ingestion must produce a faithful mirror of the source in *RAW*, and all business logic lives in dbt. This separation has a direct consequence for the Notion project: an ingestion *must not* alter the shape of the data, on pain of breaking the downstream dbt models.

2.2 Ingestion frameworks: from the in-house operator to the declarative framework

Three families of approaches coexist for extracting a source and loading it into a warehouse.

In-house operators. Historically, Qonto wrote one *Airflow operator* per source type (for example a *NotionToS3Operator*). This approach gives full control but places on the team the entire cross-cutting logic: pagination, error retries, typing, incremental loading, code that must be maintained and that is a recurring source of bugs.

Connector platforms. Tools such as Airbyte [5] provide a catalogue of ready-made connectors. They reduce the code to write but introduce heavy infrastructure (a service to operate) and impose their own destination formats and conventions, which complicates parity with an existing system.

“Library” EL frameworks. A more recent approach embeds ingestion as a *library* inside a plain Python program. The *dlt* (*data load tool*) framework [2] is a representative: the developer declares *resources* (Python generators yielding records) and a *pipeline* towards a *destination*

(Snowflake, say); the framework automatically handles API pagination, *normalisation* of semi-structured data, file *staging* and loading, including incremental loads and schema tracking. dlt thus offers the best of both worlds: little in-house code, yet without the infrastructure of a connector platform. At Qonto, dlt was already in production for other sources (LinkedIn and Facebook advertising), but with a *data-lake* destination (Iceberg); using a native Snowflake destination was a first (Chapter 3).

2.3 Orchestration and infrastructure coupling

Airflow [4] models a pipeline as a directed acyclic graph (DAG) of tasks. A structuring design choice is how the *code* and *dependencies* of the tasks are packaged. A *monorepo* with a Docker image shared by all DAGs simplifies deployment but creates strong coupling: any library upgrade potentially affects *all* pipelines. This is the *blast radius*: the more a dependency is shared, the wider the risk induced by changing it. Software engineering literature and practice recommend, to contain this risk (*dependency isolation*) for example by running each job in its own container, via Airflow's `KubernetesPodOperator` or an equivalent pattern. Qonto's *Job Orchestrator* (Chapter 1) applies exactly this principle. This coupling to the shared image is the common thread of the first two projects, formalised in §2.7.

2.4 Machine-to-machine authentication

Making two systems talk to each other without human intervention raises the question of authentication. Several mechanisms exist, of increasing stringency.

Personal access tokens (PAT). A *Personal Access Token* is a long-lived secret tied to an account. Simple to implement, it has two major flaws: its *rotation* (renewal before expiry) is a recurring manual task and a source of incidents, and its nature as a durable secret increases the exposure surface. The Tableau project (Chapter 4) started from managing fourteen annually rotated PATs.

Short-lived signed tokens (JWT). The *JSON Web Token* standard [8] lets a client *mint* its own signed token, with a very short lifetime, from a single secret. The *JWT bearer* profile of OAuth 2.0 [9] formalises exchanging that assertion for a session token. Tableau exposes this mechanism through its *Connected Apps* [7]: a single secret, never transmitted over the network, replaces the multitude of PATs; each run mints a token valid for a few minutes, with least-privilege *scopes*. This shift from a durable shared secret to a short-lived per-run token illustrates two best practices: *least privilege* and *reducing the lifetime of secrets*.

Mutual certificate authentication (mTLS/PKI). In exchanges with public administrations, authentication often relies on a public-key infrastructure (PKI) and *mutual TLS*: the client presents an X.509 certificate issued by a trusted authority. This is the model imposed by the Dutch Digipoort gateway via a PKIoverheid certificate, over an FTPS channel [10] and not to be confused with SFTP (Chapter 5).

A cross-cutting point across these mechanisms is *secrets management*: no secret should be written in clear text in the code or the image. Qonto uses AWS SSM Parameter Store (encrypted), where only the parameter *name* appears in the configuration, the value being resolved at runtime.

2.5 Warehouse loading: staging, COPY and atomic swap

Efficiently loading large volumes into Snowflake follows a well-established pattern [6]: drop the files on object storage (S3), then run a bulk `COPY INTO` command. The question of *visibility* then arises while a table is being fully reloaded in “replace” mode. A naive approach (truncate then re-insert) leaves a window during which the table returns zero rows. The *atomic swap* technique solves this: load into a *staging* table, then exchange the two tables with an instantaneous metadata operation (`SWAP`), so that a reader sees the old version until commit, then the new one with no partial state. Two subtleties, central to Chapter 3, accompany this pattern: Snowflake’s `SWAP` operation *also exchanges the grants* between the two tables.

2.6 Regulatory reporting and data transmission

Finally, part of the internship falls under *compliance*. The European CESOP directive [11] has, since 2024, required payment service providers to report their cross-border payments quarterly, in a standardised XML format, in order to fight VAT fraud. While most countries offer a web API, some, including the Netherlands, impose a government gateway (Digipoort) with a file-based exchange protocol and certificate authentication. Building such a chain is a problem of *systems integration* and *state machines* (submission, then asynchronous reconciliation of acknowledgements that may span several weeks) more than a data problem in the classical sense.

2.7 Problem statement and positioning

From this state of the art, the internship’s common problem emerges. In the first two projects, the business logic was not itself faulty; the true source of fragility was the *coupling* to Airflow’s shared Docker image, which made any change risky and concealed failures. The adopted approach is therefore, each time: (1) *diagnose* that the real constraint is infrastructural; (2) *decouple* the job into a dedicated repository and image (via the Job Orchestrator); (3) take advantage of the decoupling to *remove* a brittle legacy mechanism and *harden* the job; all of it (4) *without interruption* for downstream consumers. The third project, regulatory in nature, applies the same demand for rigour and industrialisation to a brand-new chain. The following chapters unfold this approach project by project.

Migrating Notion ingestion to dlt

This first project fully illustrates the approach outlined in §2.7: start from a production incident, trace it back to the infrastructure cause, decouple, then harden, all without interruption. It is also the project where the real difficulty turned out to be very different from the apparent one.

3.1 The legacy

Several business tables come from Notion databases (reference data, bug-tracking boards, control scopes, etc.). Their ingestion lived *inside* qonto-airflow, as a chain of three in-house operators:

`NotionToS3Operator → S3ToSnowflakeOperator → SwapSnowflakeTablesOperator`

The first operator (about 300 lines) extracted the whole Notion database and wrote a CSV to S3; the second loaded that CSV into a temporary table via `COPY INTO`; the third performed an atomic swap between the temporary table and the production table. Each table was described by a data contract (Qcontract), a YAML declaring fields, types and primary key, which the operator read to know what to extract and how to type it. The `notion-client` library was installed *in Airflow's shared base image*.

3.2 The trigger: a silent-truncation incident

In early 2026, the `notion-client` library was bumped from 0.9.0 to 3.0.0 in the base image. This bump introduced a breaking API change: the `databases.query` endpoint was replaced by `data_sources.query` (Notion having split the notion of a “database” into one or more “data sources”). But the old, now-deprecated endpoint **silently capped results at 300 rows** (six pages of fifty) by returning `has_more=false` *without raising an error*. The result: a *silent data truncation* in production, undetected by the mechanisms in place.

Beyond the one-off bug, two *structural* problems were identified during the value analysis:

1. **Strong coupling** to the shared image: bumping `notion-client` is risky (blast radius = all DAGs) and slow.
2. **No resilience** to silent failures: neither row-count validation nor alerting on truncation.

The second point is the most instructive: the problem was not merely that a figure was wrong, but that the system was *unable to detect it*. The right answer therefore could not be a simple one-off fix.

3.3 Options considered and decision

Three approaches, all based on decoupling from the base image, were compared (Table 3.1).

Approach	Principle and trade-off	Chosen
Job Orchestrator (rewrite)	Code and dependencies in a dedicated repository. Requires <i>rewriting by hand</i> the whole <code>extract</code> → S3 → COPY → swap chain.	No
Custom providers (airflow-kube-pod-jobs)	One image per job, but a <code>KubernetesPodOperator</code> <i>still</i> has to be written and maintained in <code>qonto-airflow</code> : the orchestration-side coupling remains.	No
dlt (qonto-python-dlt)	EL framework already in production, same Job Orchestrator pattern, <i>but</i> pagination, S3 staging, COPY INTO, atomic swap and load auditing provided <i>natively</i> .	Yes

Table 3.1: Comparison of the three decoupling approaches for Notion ingestion.

The decisive argument: the rewrite approach would have had me re-code *by hand* exactly what dlt already does; including pagination, which is precisely the fix for the truncation bug. dlt brought the *same decoupling* with *less in-house code* and *free resilience*. The only net-new to take on was the, at Qonto unprecedented, use of dlt’s Snowflake destination.

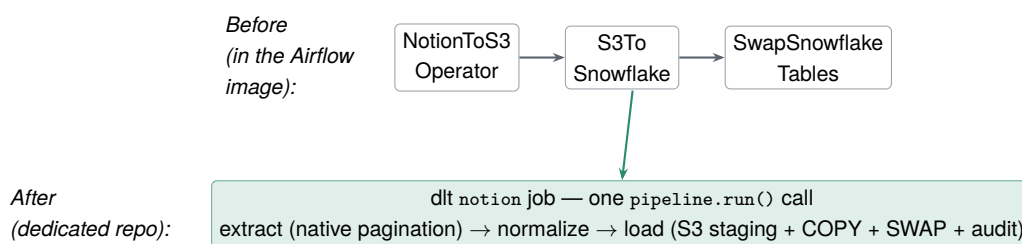


Figure 3.1: Before / after: three in-house operators coupled to the Airflow image, replaced by a single decoupled dlt job.

3.4 Implementation: the dlt architecture

The job lives in `qonto-python-dlt/jobs/notion/`, split into tested modules (`main`, `contracts`, `helpers`, `formatters`, `naming`, `governance`). The entry point is `python -m jobs.notion.main -yaml <s3://...contract.yaml>`: *one invocation = one contract = one table*.

The pivotal function, `build_resource_from_contract()`, is *pure* (no I/O): it reads the YAML contract and derives from it the list of fields, the dlt *column hints* via a `yaml_to_dlt_columns()` translator, the table name and the primary key (the contract’s, otherwise falling back to `id`), then builds the dlt resource, applying those hints with the “replace” write disposition.

Everything then starts from *a single call*, `pipeline.run()`, monolithic from the outside but internally chaining three phases:

1. **Extract:** the resource queries `/v1/data_sources/{id}/query` via dlt’s REST client and its paginator. *Pagination is delegated to the library*, which stops on the documented

has_more=false. **This is the underlying fix for the silent truncation:** an in-house paginator may terminate incorrectly and truncate silently. Records are buffered then written to local JSONL and nothing touches Snowflake yet.

2. **Normalize:** dlt re-reads the JSONL, applies the contract’s column hints, and rewrites the files. The format is forced to JSONL so that semi-structured columns land as *native* VARIANT/ARRAY.
3. **Load:** the only phase that talks to Snowflake. dlt generates the SQL sequence of Listing 3.1.

```

1 -- 1) staging table typed by the contract's column hints
2 CREATE TABLE IF NOT EXISTS RAW.NOTION_STAGING.<TABLE> (...);
3 -- 2) bulk load from the JSONL on S3 (external stage)
4 COPY INTO RAW.NOTION_STAGING.<TABLE> FROM @<s3_stage>/... FILE_FORMAT=(TYPE=JSON);
5 -- 3) atomic swap (staging-optimized + enable_atomic_swap)
6 ALTER TABLE RAW.NOTION.<TABLE> SWAP WITH RAW.NOTION_STAGING.<TABLE>;
7 -- 4) load audit row
8 INSERT INTO RAW.NOTION._DLT_LOADS (load_id, schema_name, status, inserted_at) VALUES (...);

```

Listing 3.1: SQL sequence generated by dlt during the load phase (schematic).

A methodological point I made sure to understand precisely: *I write none of these queries*. dlt produces them from the destination configuration (atomic swap, “staging-optimized” strategy) and the column hints. My work consisted in *choosing the right safe defaults* and *constraining* the framework, not in rewriting the plumbing.

3.5 The real challenge: byte-for-byte parity with the legacy

The dlt pipeline was the easy part. The central technical difficulty was elsewhere: **producing output identical to the legacy system’s**, so that no downstream dbt model would break at cutover time. Left to its defaults, dlt would have drifted along several axes; I had to constrain it everywhere (Table 3.2).

Axis	Constraint imposed for parity
Column naming	dlt normalises to <i>snake_case</i> (strips the trailing <code>_</code> , collapses <code>__</code>). I wrote a custom naming convention (<code>naming.py</code> , imposed before importing dlt) that reproduces exactly the legacy cleaning, transliterates accents/emojis and <i>suffixes Snowflake reserved words with a <code>_</code></i> (table, on, start...).
Types	Contract → dlt-hints translator routing first by the <i>physical type</i> (ODCS).
Values	Coercion by Notion property type: dates reduced to the local day, timestamps as local wall-clock time (offset stripped), integers rounded, decimals as <code>NUMBER(38,6)</code> , semi-structured kept native.
Nullability	All columns forced to <i>nullable</i> (except the primary key), because the legacy load made everything nullable.
S3 layout	Same date partitioning as the legacy, preserving load history.

Table 3.2: The five parity axes constrained to make the cutover invisible downstream.

This is the project’s key lesson: the real difficulty was not “using dlt”, but *guaranteeing zero drift* of schema or value relative to production, so that the cutover would be transparent to the hundreds of downstream dbt models. Each constraint in Table 3.2 corresponds to a real parity bug, diagnosed and then fixed, often at the cost of a fine investigation of both systems’ behaviour.

3.6 Two mechanisms re-implemented around dlt

dlt provides pagination, staging and the swap “for free”, but it does not reproduce two behaviours of the legacy swap that had to be preserved. I coded them around `pipeline.run()`.

(a) Grant preservation on the swap. Snowflake’s `SWAP WITH` operation *also exchanges grants*: after the swap, consumer roles (dbt, BI tools) would lose access on every run. I reproduced the legacy operator’s behaviour: read the grants (`SHOW GRANTS`) *before* the load, then *re-apply* them one by one *after* the swap.

(b) Schema reconciliation on contract drift. The “replace” disposition replaces *rows*, not the *schema*: dlt never re-types an existing column. If a contract re-types a field (for example `text` → `array`), dlt would keep the old type indefinitely. I wrote a drift detector (`DESC TABLE` compared to the contract’s expected types) which, on a re-typing or a removed column, switches the run to a rebuild mode; otherwise the interruption-free atomic swap is preserved. The mechanism is *fail-safe by default* (any introspection error → no destructive rebuild) and *self-resolving*.

3.7 The zero-downtime production cutover

The trickiest part was not writing the pipeline, but *cutting over production* without breaking the dbt consumers or the Airflow sensor logic. I achieved it through a *sequence of backward-compatible PRs* in `qonto-airflow` (Figure 3.2).

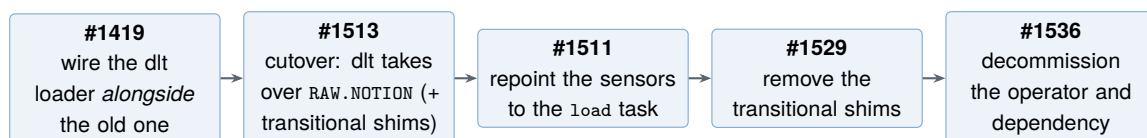


Figure 3.2: Zero-downtime cutover sequence, from wiring in parallel to decommissioning.

The detail that makes the difference: the sensors always targeted `raw.notion.<table>`, with table name and cadence *unchanged* so downstream saw nothing happen. That is the very definition of a zero-downtime migration. The final step removed the legacy operator (about 300 lines) and its tests (about 550 lines), an orphaned Airflow *pool*, an obsolete environment *mapping*, and above all the `notion-client` dependency from the base image.

Results and evaluation

- **Silent truncation eliminated by construction:** pagination delegated to the library and a `_dlt_loads` audit table on every load.

- **Coupling removed:** no more `notion-client` dependency in the Airflow base image.
- **Cutover with no downtime and no dbt regression:** byte-for-byte parity verified (names, types, nullability, time zones, S3 layout).
- **Reduced debt:** removal of the three-operator chain, the orphaned pool and the associated tests (on the order of 850 lines removed on the `qonto-airflow` side).
- **Added robustness:** grants preserved on the swap, schema drift auto-reconciled, a dedicated per-module test suite.

My contribution

I carried this project end to end: incident analysis, comparison of approaches, full development of the dlt job and its modules, resolution of the five parity axes, re-implementation of grants and schema reconciliation, and steering of the cutover PR sequence into production (#1419 to #1536). I also spotted two latent bugs in the legacy code while reading it (a loop variable used outside its loop, and an error on an empty Notion database). Ownership of the system was then transferred to the Data Engineering team.

Migrating the Tableau DAGs and JWT authentication

The second project applies the same decoupling approach as the first, but with a different root cause: here, the central problem is *authentication*. It shows how a seemingly minor constraint (tokens to renew) in fact forces a redesign of the execution architecture.

4.1 The legacy

Two families of Tableau pipelines lived in `qonto-airflow`:

- **Refresh:** a DAG `daily_refresh_tableau_workbooks` (03:00 UTC) recomputed **62 workbooks** every night, one task per workbook, each gated by sensors on the dbt models that feed it (about 140 deduplicated sensors). The priority (*P1*) workbooks are critical and page the on-call rotation on failure.
- **Metadata loading:** four DAGs queried Tableau’s *Metadata* GraphQL API (workbooks, tables, views, revisions) into Snowflake via the `extract → S3 → Snowflake` chain.

4.2 The central problem: authentication

The whole thing relied on **fourteen personal access tokens** (PATs): ten for refresh and four for metadata *expiring every year* and tracked *by hand* in a Notion table. A missed rotation broke the nightly refresh. To stay under the *concurrent-session limit per token* imposed by Tableau, the refresh used a *pool of ten tokens* managed by an XCom mechanism detailed below. Finally, the version of the Tableau client (*Tableau Server Client*, TSC) embedded in the Airflow image was *too old* to support *Connected Apps*: the move to JWT was therefore *blocked* as long as the dependency stayed in the shared image. We find here, exactly, the coupling identified in Chapter 2.

4.2.1 The legacy XCom mechanism

This mechanism deserves an explanation, as it illustrates a subtle piece of technical debt. XCom is the communication channel between Airflow tasks, designed to be *immutable* (append-only). The legacy code diverted it to implement a token pool:

1. an operator pushed, at the start of the DAG, the list of the ten available connection names (the “whiteboard” of free tokens);
2. one sensor per workbook, running in a *single-slot Airflow pool* (a *mutex*), read the list, *removed* a token and *rewrote* the shortened list;

3. the refresh operator used the acquired token;
4. a release operator returned the token at the end of the run (even on failure).

Why a single-slot mutex? Because the 62 acquisitions run in parallel and all perform a *read–modify–write* cycle on the same list: without serialisation, two tasks would take the same token (a *race condition*). The mutex guarantees a single mutation at a time, to be distinguished from the *parallelism of ten* (the number of tokens). Above all, to mutate an XCom value mid-run, the legacy code *wrote directly to the xcom table* of Airflow’s metadata database which is a mutable state hacked into the heart of the scheduler, brittle and invisible to Airflow. This is the *hack* that the migration removed.

4.3 Solution: dedicated pods and a JWT Connected App

The solution combines decoupling and an authentication redesign:

- **Migration of both pipelines** to `qonto-python-data-engineering (jobs/tableau/)`, run as `CustomJobKubernetesPodOperator` — one pod per workbook, thus *outside* the Airflow image. The `apache-airflow-providers-tableau` provider is removed from `qonto-airflow`.
- **Replacement of the 14 PATs by a single JWT Connected App.** A `make_jwt()` function mints, on each run, an HS256 token (exp at 5 minutes, a unique anti-replay identifier, least-privilege scopes), signed by a single secret resolved from AWS SSM. The application identifiers are public; only the secret value is confidential and is never transmitted or committed. **One secret replaces fourteen PATs.**
- **Full removal** of the XCom pool, the mutex and the associated operators. The refresh DAG goes from *five tasks per workbook to two* (dbt sensor → pod); metadata from *three tasks to one pod* (Figure 4.1).
- The configuration of the 62 workbooks is *driven by Python modules* and the refresh `schedule.yaml` is *generated*, with a continuous-integration test keeping generation and modules in sync.

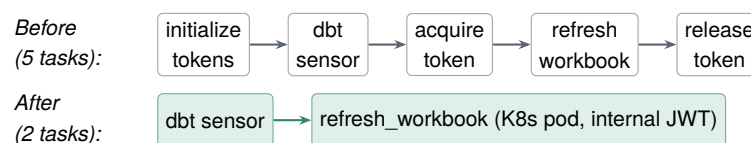


Figure 4.1: Refreshing a workbook: before and after.

4.4 How the JWT works

On each run, `make_jwt` encodes an HS256 token carrying the issuer (client identifier), the audience (`tableau`), the subject (service account), the requested scopes, a unique identifier and a five-minute expiry; the header references the secret identifier. This token is passed to the Tableau client, whose authentication call returns a *session token* (valid for up to 240 minutes of inactivity) used thereafter.

4.5 Difficulties encountered

Three concrete obstacles had to be overcome, each instructive.

The jobs API blocked under JWT. Tracking a refresh's progress used the jobs API, which returns a 401 error for a session issued by a Connected App. *Fix:* detect the end of the refresh by *polling* the workbook's last-update timestamp (`updated_at`) every 30 seconds until it changes.

Refresh-pod starvation. An initial global cap made the slots *shared* between the 140 dbt sensors and the pods: a ready pod could wait up to 90 minutes. *Fix:* a *dedicated Airflow pool* (ten slots) bounding *only* the refresh pods.

Idempotency on 409 and 429. If a refresh is already queued (code 409, for example triggered manually from the UI), we do not re-trigger it: we wait for the one in flight. Rate limiting (code 429) was handled with a back-off; it is now avoided upstream by the pool of ten.

Important nuance: the real constraint is server-side

Tableau Cloud caps concurrent extract refreshes at *ten* per site. After migration, I *keep* this limit of ten, but via a *simple Airflow pool* rather than through token scarcity. What disappears: token scarcity, the XCom mutex and manual rotation. In other words, the migration does not remove the server limit, it removes the *brittle plumbing* that simulated it.

4.6 Decisions and trade-offs

Why not simply bump the TSC in the Airflow image? Because the image is *shared by all DAGs*: bumping the Tableau client risked breaking other providers and coupled a product library to the lifecycle of everyone's image. Moving the dependency into a dedicated repository and pods unblocks the JWT *and* reduces the risk surface to the Tableau perimeter alone.

Accepted trade-offs: one more repository, image and deployment pipeline, plus a *cross-repo contract*. The gain; JWT unblocked, zero rotation, removal of the XCom hack far outweighs it.

Results and evaluation

- **One secret instead of fourteen PATs**, and *zero manual rotation*: the "missed rotation → broken refresh" incident class disappears.
- **Simplification**: refresh from 5 to 2 tasks per workbook, metadata from 3 tasks to 1 pod; removal of the token pool, the mutex and the XCom mutation hack.
- **Decoupling**: Tableau provider removed from the base image; execution in isolated pods.
- **Reduced debt**: on the order of 880 lines of legacy code removed.
- **Cleanly bounded concurrency**: still capped at ten (Tableau Cloud limit), but through an explicit pool.

My contribution

I led the entire migration: design and implementation of the JWT authentication and its tests, porting of the refresh and metadata pipelines to pods, configuration-driven generation of the 62 workbooks, resolution of the three difficulties above, and decommissioning of the legacy runtime on the `qonto-airflow` side (PR #1633/#1634). In addition, I tooled the onboarding of a new workbook via a *skill* that validates, generates the configuration and opens the PR; a contribution to the team's automation culture.

CESOP Netherlands regulatory transmission chain

The third project differs from the first two: it is not about migrating an existing system, but about *building a brand-new* regulatory transmission chain. It draws more on systems and protocol engineering than on data engineering.

5.1 Regulatory context

Since 1 January 2024, the European *CESOP* directive [11] has required every payment service provider (PSP) to report its cross-border payments *quarterly*, in order to fight VAT fraud in cross-border e-commerce. The regulatory trigger: as soon as a PSP executes more than 25 cross-border payments to the same payee in a quarter, all those payments must be reported; authorities then cross-check the data between member states. As Qonto is a PSP, this reporting is an already-running process for the countries where it operates. The new element: the opening of a Dutch IBAN, operational since December 2025, which makes the Netherlands a new reporting country, with a first affected quarter (Q4 2025). And *the Netherlands does it differently from every other country* which is precisely the subject of this project.

5.2 The Dutch specificity

Where other countries accept submission via a web API (HTTPS + API key or OAuth), the Netherlands imposes a government gateway, **Digipoort** (operated by the agency Logius), with an entirely different technical stack (Table 5.1).

Element	Other countries (ES/IT/FR/DE)	Netherlands
Submission point	National tax portal	The Digipoort gateway
Authentication	API key / OAuth	PKIoverheid certificate (X.509), mTLS
Transmission	HTTPS POST	FTPS + a .meta companion file
Feedback	Direct API response	Files dropped into an out/ folder to be polled (hours to weeks)

Table 5.1: The gap between the usual CESOP submission and the Dutch route (Digipoort).

The most important design point is the distinction between *the what* and *the how*. *The what*: the XML content (payments, payees, amounts) is the *payload*, produced by an existing generator

in the Analytics Engineering team; it *arrives as an input* to my project and follows the standard CESOP format. *The how*: authenticate, attach a `.meta`, transmit, reconcile the acknowledgements is my scope. I build the pipes, not the water flowing through them.

5.3 Architecture: two DAGs and a state table

The “submission → acknowledgement” loop is *asynchronous* and may span hours to weeks: a single task therefore cannot stay blocked waiting. The design splits the work into two DAGs, linked by a Snowflake state table (Figure 5.1).

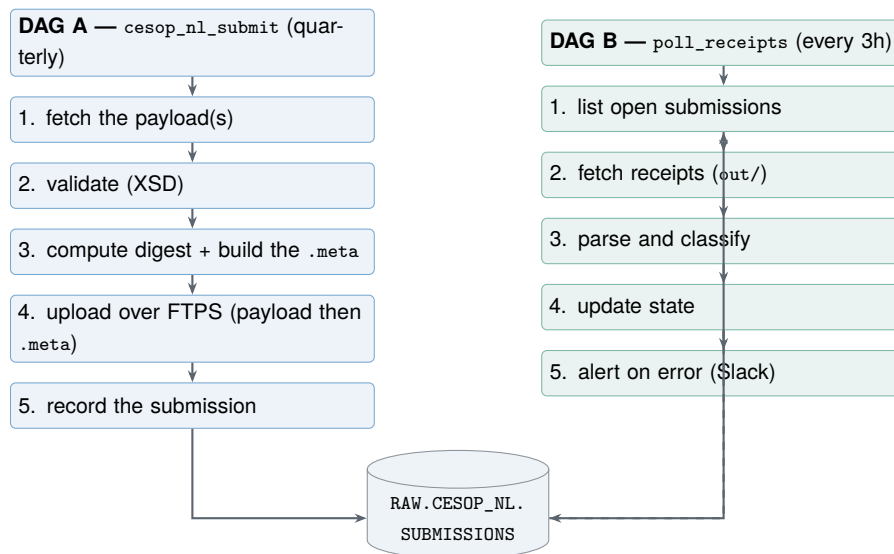


Figure 5.1: Two-DAG architecture: a quarterly submitter and a three-hourly receipt poller, linked by a state table.

The `RAW.CESOP_NL.SUBMISSIONS` table holds one row per submission and tracks its lifecycle: transport status, technical validation (a few hours), then business validation (days to weeks). DAG A inserts the rows; DAG B updates them as acknowledgements arrive.

5.4 Implementation

The job lives in `qonto-python-data-engineering/jobs/cesop_nl/`, following the repository’s conventions (`config.py` with Pydantic settings, `main.py`, `queries/`) and organised by responsibility:

- `transport/`: the **FTPS/mTLS** client to Digipoort (connect, upload the file pair, list and download acknowledgements) and the `.meta` **builder** compliant with the `metabestand.xsd` schema;
- `submit/`: DAG A’s flow (fetch, validate, digest, `.meta`, upload, record);
- `poll/`: DAG B’s flow, with an **acknowledgement parser** that classifies the four artefact types in the `out/` folder into a uniform result, and the archival of raw receipts on S3;
- `state/`: the Snowflake state-table repository and its schema.

A few notable implementation points. The `.meta` file is a small XML that accompanies each payload: it carries the reference identifier (equal to the exact payload filename), the sender, the

fixed receiver for CESOP and the size. The upload order is **strict**: the payload first, *then* the `.meta` in the same session. It is the arrival of the `.meta` that triggers processing on the Digipoort side. Trusting Logius's server certificate required explicitly installing the certificate-authority chain. Rejection alerts are surfaced on Slack in *Block Kit* format. Finally, *resubmission* handling was implemented: a rejected delivery must be resent as a *fresh initial submission* (not as a correction), with a file-identifier increment and dated archival of the acknowledgements.

5.5 Sticking points resolved

The analysis phase resolved several uncertainties, by reading the official specifications (Logius FTPS interface, the tax authority's "dialogue" document, the PSPNL manual):

- **"Signing" = computing a digest, not signing the XML.** The certificate is used for TLS authentication; integrity is provided by an optional *SHA-512* digest in the `.meta`. No XML signature (XMLDSig) is required which was a major simplification confirmed with the regulatory side, which avoids a heavy cryptographic dependency.
- **A quarter may produce several files** (a 100 MB split rule): the job handles a *list* of payloads, not a single file.

My contribution

I handled the analysis (reading and synthesising the official specifications written in Dutch language, resolving the sticking points) then the design and implementation of the entire transport chain: package structure, `.meta` builder, FTPS/mTLS client, acknowledgement parser, state model, and both end-to-end flows, through to scheduling, S3 archival and Slack alerts. I collaborated with the *Regulatory* team (specifications, certificate) and *Analytics Engineering* team for the dependencies outside my scope.

Assessment and skills acquired

This chapter steps back from the three projects to draw out the skills acquired and the common methodological pattern, as recommended by the assessment guidelines.

6.1 A common pattern: diagnose, decouple, harden

In hindsight, the three projects share the same outline, summarised in Table 6.1. The first two are migrations: the true source of fragility was not the business logic but a shared infrastructure dependency, and the right answer was to *decouple* before *hardening*, without ever interrupting the service. The third, regulatory in nature, applies the same rigour to a brand-new chain.

Axis	Notion project	Tableau project
Triggering constraint	<code>notion-client</code> 0.9→3.0 in the base image: breaking API + silent truncation	Tableau client too old in the base image: JWT blocked
Legacy fragility	No truncation detection	14 manually rotated PATs + token pool + XCom hack
Decoupling	To <code>qonto-python-dlt</code> (dlt Snowflake destination)	To <code>qonto-python-data-engineering</code> (K8s pods)
Real difficulty	Byte-for-byte parity + zero-downtime cutover	Unblocking the JWT by moving the dependency out + avoiding pod starvation
Resilience gained	Native pagination, load auditing, atomic swap	A single secret, short-lived token, no more rotation

Table 6.1: Summary of the pattern common to the two migrations.

6.2 Technical skills

This internship let me consolidate and acquire a set of production-grade data-engineering skills:

- **Ingestion and warehousing:** command of the EL paradigm, of the dlt framework (resources, pipelines, column hints, write dispositions), and of Snowflake loading mechanisms (S3 staging, `COPY INTO`, atomic swap, grant and schema-drift handling).
- **Orchestration:** designing Airflow DAGs, decoupling patterns via Kubernetes pods, sensors and dependency management, concurrency pools.
- **Security and authentication:** implementing JWT authentication (Connected App), understanding the PAT, OAuth and mTLS/PKI models, and secrets management through an encrypted parameter store.

- **Software quality:** per-module unit tests, continuous integration, linting, configuration validation, and tooling (a *skill*) to automate recurring tasks.
- **Systems integration:** transport protocols (FTPS/mTLS), XML formats and schema validation, state machines and asynchronous reconciliation (CESOP project).

6.3 Methodological skills

Beyond the technical side, the internship reinforced a way of working: *trace problems back to the root cause* before coding (both migrations were born from a refusal to treat a symptom); *make trade-offs explicit* rather than suffer them (choice of dlt, the Tableau server limit, signing reduced to a digest for CESOP); *design for reversibility and zero interruption* (backward-compatible PR sequences); and *measure* the effect of changes (lines removed, tasks reduced, an incident class eliminated). The team's *Value Analysis / Value Engineering* approach served as a guiding thread for each of these points.

6.4 Transferable skills

I gained autonomy and *ownership*: taking a high-impact topic and carrying it into production entails real responsibility, especially for critical pipelines that page the on-call rotation. Collaboration was essential: demanding code reviews, exchanges with different teams by documenting decisions, sticking points and trade-offs on Notion, both to gain buy-in in review and to transfer ownership of the systems to the team.

6.5 Value for the company

The deliverables carry direct and lasting value: the removal of two incident classes (silent truncation; missed token rotation), a significant reduction of legacy code and of coupling to the shared image, and the establishment of a hitherto non-existent regulatory chain. Beyond that, the patterns established (dlt's first Snowflake destination, a zero-downtime migration template, reusable JWT authentication) are references for the team's future migrations.

Conclusion and perspectives

This end-of-studies internship, carried out within Qonto's Data Platform team, aimed to harden and industrialise business-critical data pipelines. The problem posed in the introduction on how to harden pipelines with no service interruption when the real source of fragility is a shared infrastructure dependency, found a coherent answer across three projects.

The first two, migrating Notion ingestion to dlt and migrating the Tableau pipelines to JWT authentication, validated the same approach: diagnosing that the coupling to Airflow's shared Docker image was the true constraint, decoupling the job into a dedicated repository and image, then removing a brittle legacy mechanism. In both cases the decisive difficulty was not writing the new code, but *achieving the cutover without downstream consumers noticing*: byte-for-byte parity with production for Notion, mastery of concurrency and of the new authentication for Tableau. The third project, the CESOP transmission chain to the Dutch Digipoort gateway, applied the same rigour to a systems-integration and compliance problem.

In terms of results, this work eliminated two incident classes by construction, removed all manual secret rotation, took a risky dependency out of the shared image, and appreciably reduced legacy code and the number of Airflow tasks, while giving the company a new regulatory capability.

Several **perspectives** naturally extend these projects. For Notion ingestion, adding *freshness tests* and volume checks would complement the resilience already gained, and dlt's Snowflake destination could be generalised to other sources currently carried by in-house operators. For Tableau, the JWT authentication template and the workbook-onboarding tooling can be reused beyond the current perimeter. For CESOP, the next step is monitoring the first real delivery through the business-validation window that may span several weeks. Finally, the *zero-downtime decoupling* pattern established during this internship is a template transferable to the platform's future migrations.

On a personal level, this internship was a full immersion in production-scale data engineering: it taught me to favour root-cause diagnosis over symptom fixing, to explicitly own engineering trade-offs, and to measure the value of my work. All reflexes I consider the main takeaway of this experience.

Bibliography

- [1] M. Kleppmann, *Designing Data-Intensive Applications*, O'Reilly Media, 2017.
- [2] dltHub, *dlt — data load tool: Documentation*, 2026. <https://dlthub.com/docs>
- [3] dbt Labs, *dbt Documentation*. <https://docs.getdbt.com>
- [4] Apache Software Foundation, *Apache Airflow Documentation*. <https://airflow.apache.org/docs/>
- [5] Airbyte, *Airbyte Documentation*. <https://docs.airbyte.com>
- [6] Snowflake Inc., *COPY INTO <table> — Snowflake Documentation*. <https://docs.snowflake.com/en/sql-reference/sql/copy-into-table>
- [7] Tableau (Salesforce), *Configure Tableau Connected Apps with Direct Trust*. https://help.tableau.com/current/online/en-us/connected_apps.htm
- [8] M. Jones, J. Bradley, N. Sakimura, *JSON Web Token (JWT)*, RFC 7519, IETF, May 2015. <https://www.rfc-editor.org/rfc/rfc7519>
- [9] M. Jones, B. Campbell, C. Mortimore, *JSON Web Token (JWT) Profile for OAuth 2.0 Client Authentication and Authorization Grants*, RFC 7523, IETF, May 2015. <https://www.rfc-editor.org/rfc/rfc7523>
- [10] P. Ford-Hutchinson, *Securing FTP with TLS*, RFC 4217, IETF, October 2005. <https://www.rfc-editor.org/rfc/rfc4217>
- [11] Council of the European Union, *Council Directive (EU) 2020/284 amending Directive 2006/112/EC as regards introducing certain requirements for payment service providers (CESOP)*, 2020. <https://eur-lex.europa.eu/eli/dir/2020/284>
- [12] R. Dillet, *French B2B fintech Qonto reaches 600,000 customers and files for a banking license*, TechCrunch, July 2025. <https://techcrunch.com/2025/07/02/french-b2b-fintech-qonto-reaches-600000-customers-files-for-banking-license>
- [13] Wikipedia, *Qonto*, accessed 2026. <https://en.wikipedia.org/wiki/Qonto>

Detailed presentation of Qonto

Qonto is a European neobank for professionals (freelancers, very small businesses and SMEs), founded in 2017 by Alexandre Prot and Steve Anavi and operated by the company Olinda SAS [13]. Its offering covers the business account, payment cards, transfers, invoicing and pre-accounting and financial-steering tools. At the time of the internship, the company reports on the order of 600,000 customers, operates in eight European countries (France, Germany, Italy, Spain, Austria, Belgium, the Netherlands, Portugal) and states the ambition of reaching two million customers by 2030 [12]. On the regulatory side, Qonto operates as a payment institution and has begun the process of obtaining a banking licence. This payment-service-provider status underpins the regulatory-reporting obligations discussed in Chapter 5.

Technical environment and repositories

The projects span several repositories:

`qonto-airflow`

Airflow monorepo (declarative DAG generation, shared Docker base image). Target of the decommissionings and production cutovers.

`qonto-python-dlt`

dlt-based ingestion repository (Snowflake destination). Home of the new Notion job.

`qonto-python-data-engineering`

Job Orchestrator: Python jobs run in Kubernetes pods. Home of the Tableau and CESOP jobs.

`data-contracts`

Data contracts (Qontract, YAML) describing the schema of ingested tables.

Technical stack: Apache Airflow (orchestration), Snowflake (warehouse), Amazon S3 (staging), dbt (transformation), Kubernetes (isolated execution), AWS SSM Parameter Store (secrets), Python 3.12, `uv` (dependency management).

Main contributions (Pull Requests)

As concrete evidence of the work carried out, the main merged PRs, by project.

Notion project (`qonto-python-dlt`). Porting of the tested modules (#46) and wiring of the test DAGs (#47); naming conventions for parity (#55, #56, #57); JSONL loading for native VARIANT/ARRAY (#68); date and time-zone parity (#70, #79); serialisation of list/object columns (#71, #72, #73); nullability aligned with the legacy (#75); tuned retry session for long fetches (#80); reserved-keyword suffixing (#82); schema reconciliation on contract drift (#83, #84).

Notion project — cutover (`qonto-airflow`). Wiring the dlt loader in parallel (#1419); cutover to `RAW.NOTION` (#1513); repointing the sensors (#1511); removing the transitional shims (#1529); decommissioning the legacy operator and removing the `notion-client` dependency (#1536).

Tableau project (`qonto-python-data-engineering`). JWT dependencies and logic (#171, #173); metadata loader (#179, #180); refresh pipeline with generated schedule (#187); concurrency cap and 409 idempotency (#191); workbook-onboarding skill (#194); publishing the priority seed to S3 (#196); documentation (#204).

Tableau project — cutover (`qonto-airflow`). Per-task priority from `schedule.yaml` (#1593); per-DAG `max_active_tasks` (#1602); P2 priority seed from S3 (#1626); removal of the legacy refresh runtime — operator, tokens, PATs, pool (#1633, #1634).

CESOP project (`qonto-python-data-engineering`). Project blueprint (#221); `.meta` builder (#222); acknowledgement parser (#224); Snowflake state model (#225); Digipoort FTPS client (#226); submission flow — DAG A (#228); Logius CA trust (#229); polling flow — DAG B (#230).

APPENDIX D

Code excerpts

JWT authentication (Tableau project)

The function that mints the short-lived HS256 token for the Tableau *Connected App* (Chapter 4):

```
1 def make_jwt(client_id: str, secret_id: str, secret_value: str, scopes: list[str]) -> str:
2     return jwt.encode(
3         {
4             "iss": client_id,
5             "exp": datetime.now(UTC) + timedelta(minutes=5),
6             "jti": str(uuid.uuid4()), # anti-replay
7             "aud": "tableau",
8             "sub": TABLEAU_USER,
9             "scp": scopes, # least privilege
10        },
11        secret_value,
12        algorithm="HS256",
13        headers={"kid": secret_id, "iss": client_id},
14    )
```

Listing D.1: make_jwt: minting a short JWT signed by a single secret.

Naming convention for parity (Notion project)

The custom naming convention imposed on dlt to preserve production column names (Chapter 3): it merely lowercases, without touching underscores, and declares itself case-insensitive to emit unquoted DDL.

```
1 class NamingConvention(BaseNamingConvention):
2     # dlt collapses "__" ; we pick an out-of-band separator to
3     # avoid the collision with Notion column names.
4     PATH_SEPARATOR: ClassVar[str] = "_"
5
6     def normalize_identifier(self, identifier: str) -> str:
7         identifier = super().normalize_identifier(identifier)
8         return self.shorten_identifier(identifier.lower(), identifier, self.max_length)
9
10    @property
11    def is_case_sensitive(self) -> bool:
12        return False
```

Listing D.2: “Passthrough” naming convention (excerpt).